

UNIT-4

INHERITANCE, PACKAGES AND INTERFACES

4.1 BASIC OF INHERITANCE

inheritance is a fundamental feature of object-oriented programming that allows you to create new classes based on existing classes. The class that is being extended is called the superclass or parent class, and the class that inherits from the superclass is called the subclass or child class. Inheritance promotes code reuse and supports the concept of "is-a" relationship between classes.

To define a subclass and establish inheritance, you use the **extends** keyword in the class declaration. The subclass inherits all the fields and methods (except for private members) of the superclass.

EXAMPLE:

```
// Superclass
class Animal {
    protected String name;
    public Animal(String name) {
        this.name = name;
    }
    public void eat() {
        System.out.println(name + " is eating.");
    }
}

// Subclass inheriting from Animal
class Dog extends Animal {
    public Dog(String name) {
        super(name); // Calling the superclass constructor
    }
    public void bark() {
        System.out.println(name + " is barking.");    } }
```

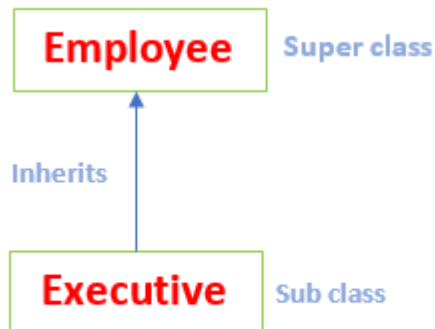
Key points to note about inheritance:

- Subclasses can access non-private members (fields and methods) of the superclass.
- Subclasses can add their own fields and methods or override the behaviour of the superclass methods.
- Constructors are not inherited but can be called using the `super()` keyword to invoke the superclass constructor.
- Java supports single inheritance, which means a class can only extend one superclass. However, it allows for multiple levels of inheritance, where a subclass can itself become a superclass for another subclass.

4.2 TYPES OF INHERITANCE

1. Single Inheritance:

In single inheritance, a subclass extends a single superclass. It is the most common type of inheritance.



Single Inheritance

EXAMPLE:

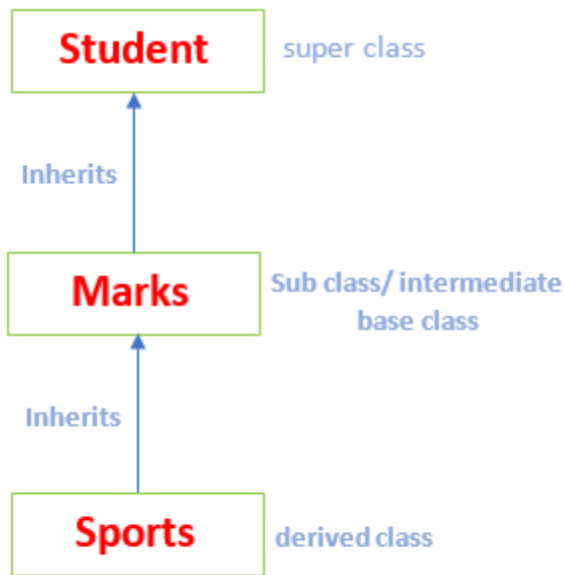
```
class Employee {
    void salary() {
        System.out.println("Salary= 200000");
    }
}

class Programmer extends Employee {
    // Programmer class inherits from Employee class
    void bonus() {
        System.out.println("Bonus=50000");
    }
}

class single_inheritance {
    public static void main(String args[]) {
        Programmer p = new Programmer();
        p.salary(); // calls method of super class
        p.bonus(); // calls method of sub class
    }
}
```

2.Multilevel Inheritance:

In multilevel inheritance, a subclass becomes the superclass for another subclass, creating a chain of inheritance.



Multi-level Inheritance

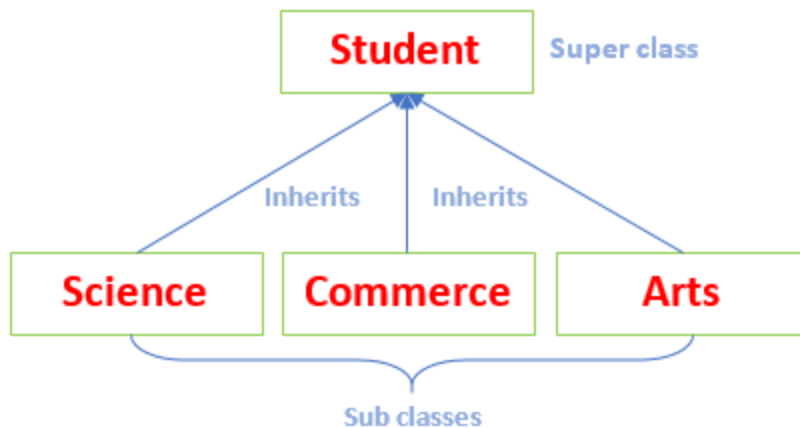
EXAMPLE:

```

class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}
}
  
```

3.Hierarchical Inheritance:

In hierarchical inheritance, multiple subclasses extend a single superclass.



Hierarchical Inheritance

EXAMPLE:

```

class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[])
{
Cat c=new Cat();
c.meow();
c.eat();
}
}
    
```

4. Multiple Inheritance (through Interfaces):

Java does not support multiple inheritance of classes, but it allows multiple inheritance of interfaces. A class can implement multiple interfaces, inheriting their method signatures.

```
interface Interface1 {
    // Interface1 members
}

interface Interface2 {
    // Interface2 members
}

class MyClass implements Interface1, Interface2 {
    // MyClass members
}
```

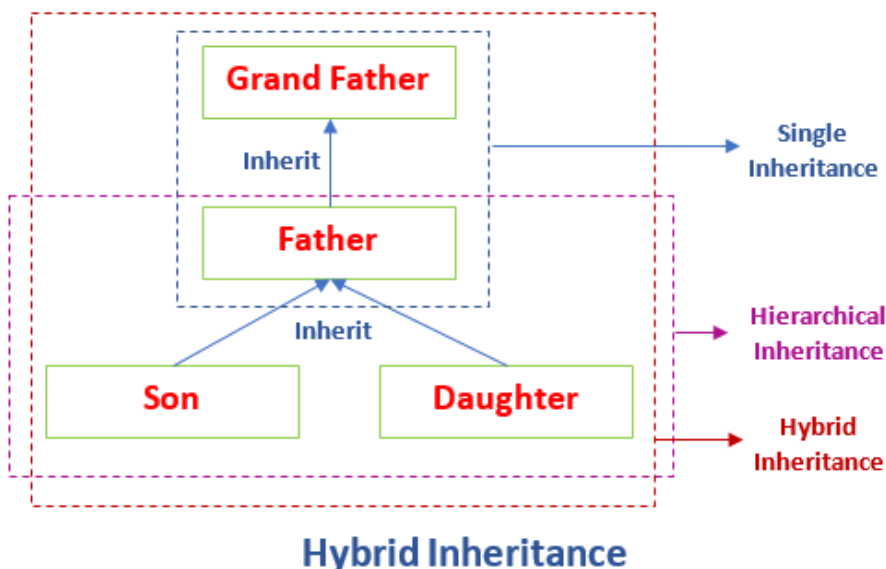
5. Hybrid Inheritance:

Hybrid inheritance combines multiple types of inheritance, such as a combination of single inheritance, multilevel inheritance, and multiple inheritance.

```
class Superclass {
    // Superclass members
}

class IntermediateSubclass extends Superclass {
    // Intermediate subclass members
}

class Subclass extends IntermediateSubclass implements Interface1, Interface2 {
}
```



4.3 METHOD OVERRIDING

Method overriding in Java allows a subclass to provide a different implementation of a method that is already defined in its superclass. It is a feature of inheritance that promotes polymorphism, allowing objects of different subclasses to be treated uniformly through their common superclass.

To override a method in a subclass, the following conditions must be met:

- The method in the subclass must have the same name and parameters (method signature) as the method in the superclass.
- The method in the superclass must be marked as public, protected, or have default (package-private) access, to be accessible to the subclass.
- The return type of the method in the subclass must be the same or a subtype (covariant return type) of the return type in the superclass. In Java 5 and later versions, you can override a method with a covariant return type.

EXAMPLE:

```
class Vehicle
{
    //defining a method
    void run(){System.out.println("Vehicle is running");}
}

//Creating a child class
class Bike2 extends Vehicle{
    //defining the same method as in the parent class
    void run(){System.out.println("Bike is running safely");
}

    public static void main(String args[]){
        Bike2 obj = new Bike2();//creating object
        obj.run();//calling method
    }
}
```

When an overridden method is called on an object, the decision on which implementation to invoke is made at runtime based on the actual type of the object (dynamic method dispatch). This allows for polymorphism, where different subclasses can have their own specific implementation of the same method.

Method overriding allows you to provide specialized behaviour in subclasses while still adhering to the common interface defined by the superclass. It is a powerful mechanism for achieving code reuse and flexibility in object-oriented programming.

4.4 SUPER KEYWORD

The **super keyword** is used to refer to the superclass or parent class of a subclass. It is commonly used in two scenarios:

Usage of Java super Keyword

- super can be used to refer immediate parent class instance variable.
- super can be used to invoke immediate parent class method.
- super() can be used to invoke immediate parent class constructor

1) Super Is Used To Refer Immediate Parent Class Instance Variable.

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

EXAMPLE:

```
class Animal
{
    String color="white";
}

class Dog extends Animal
{
    String color="black";
    void printColor()
    {
        System.out.println(color);//prints color of Dog class
        System.out.println(super.color);//prints color of Animal class
    }
}

class TestSuper1{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.printColor();
    }
}
```

Output:

black

white

2) Super Can Be Used To Invoke Parent Class Method:

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class.

EXAMPLE:

```
class Animal
{
void eat(){System.out.println("eating...");
}
}

class Dog extends Animal
{
void eat()
{ System.out.println("eating bread...");
}
void bark(){ System.out.println("barking...");
}
void work()
{
super.eat();
bark();
}
}

class TestSuper2
{
public static void main(String args[])
{
Dog d=new Dog();
d.work();
}
}
```

Output:

eating...

barking...

3) Super Is Used To Invoke Parent Class Constructor:

The super keyword can also be used to invoke the parent class constructor.

EXAMPLE:

```
class Animal
{
    Animal()
    {
        System.out.println("animal is created");
    }
}

class Dog extends Animal
{
    Dog()
    {
        super();
        System.out.println("dog is created");
    }
}

class TestSuper3
{
    public static void main(String args[])
    {
        Dog d=new Dog();
    }
}
```

Output:

animal is created

dog is created

4.5 DYNAMIC METHOD DISPATCH

Dynamic method dispatch in Java is a mechanism that allows the selection of the appropriate method implementation at runtime, based on the actual type of the object rather than the reference type. It enables polymorphism, where objects of different subclasses can be treated uniformly through their common superclass.

EXAMPLE:

```
class Phone{
    public void showTime(){
        System.out.println("Time is 8 am");
    }
    public void on(){
        System.out.println("Turning on Phone...");
    }
}

class SmartPhone extends Phone{
    public void music(){
        System.out.println("Playing music...");
    }
    public void on(){
        System.out.println("Turning on SmartPhone...");
    }
}

public class CWH {
    public static void main(String[] args)
    {
        Phone obj = new SmartPhone(); // Yes it is allowed
        // SmartPhone obj2 = new Phone(); // Not allowed

        obj.showTime();
        obj.on();
        // obj.music(); Not Allowed
    }
}
```

4.6 FINAL KEYWORD

The final keyword is used to restrict the modification or inheritance of classes, methods, and variables. When applied to different elements, it provides different behaviours:

1.Final Classes:

When a class is declared as **final**, it cannot be subclasses. It ensures that the class's implementation cannot be changed by any other class.

EXAMPLE:

```
final class finalclass
{
    void method()
    {
        System.out.println("this is final class");
    }
}

class childClass extends finalclass
{
    void method()
    {
        System.out.println("this is the childClass");
    }
}

class MainClass
{
    public static void main(String arg[])
    {
        finalclass fx = new childClass();
        fx.method();
    }
}
```

2.Final Methods:

When a method is declared as **final**, it cannot be overridden by any subclass. This is useful when you want to enforce a specific implementation of a method in a class hierarchy.

EXAMPLE:

```
class Bike{
    final void run(){System.out.println("running"); }
}

class Honda extends Bike{
    void run(){System.out.println("running safely with 100kmph"); }
    public static void main(String args[]){
        Honda honda= new Honda();
        honda.run();
    }
}
```

3.Final Variables:

When a variable is declared as **final**, its value cannot be modified once it is assigned. It becomes a constant, and the variable name is written in uppercase by convention.

EXAMPLE:

```
class Bike9{
    final int speedlimit=90;//final variable
    void run(){
        speedlimit=400;
    }
    public static void main(String args[]){
        Bike9 obj=new Bike9();
        obj.run();
    }
}
```

The **final** keyword provides immutability, security, and performance benefits. By marking a class, method, or variable as **final**, you can ensure that its behaviour cannot be modified or overridden, protecting critical functionality or values.

It's important to note that using the **final** keyword should be done judiciously. Overusing it may limit the flexibility and extensibility of your code, so it's recommended to use it when necessary, such as for constants, non-overridable methods, or classes that should not be subclassed.

4.7 ABSTRACT CLASS AND METHOD

An abstract class is a class that cannot be instantiated and is meant to serve as a blueprint for subclasses. It is declared using the abstract keyword. Abstract classes can contain abstract methods, which are declared without an implementation and must be overridden by concrete subclasses.

Here are the key points about abstract classes and methods in Java:

1. Abstract Classes:

- An abstract class is declared using the abstract keyword.
- It may contain abstract methods, concrete methods, instance variables, and constructors.
- Abstract classes cannot be instantiated directly using the new keyword.
- Subclasses of an abstract class must either implement all the abstract methods or be declared as abstract themselves.
- Abstract classes can provide common functionality and define a common interface for their subclasses.

EXAMPLE:

```
abstract class Shape
{
    abstract void draw();
}

class Rectangle extends Shape
{
    void draw(){System.out.println("drawing rectangle");
}

}

class Circle1 extends Shape
{
    void draw(){System.out.println("drawing circle");
}

}

class TestAbstraction1
{
    public static void main(String args[]){
        Shape s=new Circle1();
        s.draw();
    }
}
```

2. Abstract Methods:

- An abstract method is declared using the abstract keyword and does not have an implementation.
- Abstract methods are meant to be overridden by concrete subclasses.
- Subclasses must provide an implementation for all the abstract methods inherited from the abstract class.
- Abstract methods do not have a body and end with a semicolon.

EXAMPLE:

```
abstract class Multiply
{
    // abstract methods
    // sub class must implement these methods
    public abstract int MultiplyTwo (int n1, int n2);
    public abstract int MultiplyThree (int n1, int n2, int n3);
    // regular method with body
    public void show() {
        System.out.println ("Method of abstract class Multiply");
    }
}

// Regular class extends abstract class
class AbstractMethodEx1 extends Multiply {
    public int MultiplyTwo (int num1, int num2) {
        return num1 * num2;
    }

    public int MultiplyThree (int num1, int num2, int num3) {
        return num1 * num2 * num3;
    }

    // main method
    public static void main (String args[]) {
        Multiply obj = new AbstractMethodEx1();
        System.out.println ("Multiplication of 2 numbers: " + obj.MultiplyTwo (10, 50));
        System.out.println ("Multiplication of 3 numbers: " + obj.MultiplyThree (5, 8, 10));
        obj.show();
    }
}
```

4.8 INTERFACE

Interfaces are useful for achieving abstraction, defining contracts, and enabling polymorphism. They provide a way to define behaviour without specifying the implementation details, allowing for flexibility and code reuse. They are widely used in Java to define APIs and ensure a consistent interface across different classes.

EXAMPLE:

```
interface In1 {

    // public, static and final
    final int a = 10;

    // public and abstract
    void display();
}

// A class that implements the interface.
class TestClass implements In1 {

    // Implementing the capabilities of
    // interface.
    public void display(){
        System.out.println("Geek");
    }

    // Driver Code
    public static void main(String[] args)
    {
        TestClass t = new TestClass();
        t.display();
        System.out.println(a);
    }
}
```

4.8.1 Multiple Inheritance in Java Using Interface

EXAMPLE:

```
interface API
{
    // Default method
    default void show()
    {

        // Print statement
        System.out.println("Default API");
    }
}

// Interface 2
// Extending the above interface
interface Interface1 extends API
{
    // Abstract method
    void display();
}

// Interface 3
// Extending the above interface
interface Interface2 extends API
{
    // Abstract method
    void print();
}

// Main class
// Implementation class code
class TestClass implements Interface1, Interface2
```



```

{
    // Overriding the abstract method from Interface1
    public void display()
    {
        System.out.println("Display from Interface1");
    }

    // Overriding the abstract method from Interface2
    public void print()
    {
        System.out.println("Print from Interface2");
    }

    // Main driver method
    public static void main(String args[])
    {
        // Creating object of this class
        // in main() method
        TestClass d = new TestClass();

        // Now calling the methods from both the interfaces
        d.show(); // Default method from API
        d.display(); // Overridden method from Interface1
        d.print(); // Overridden method from Interface2
    }
}

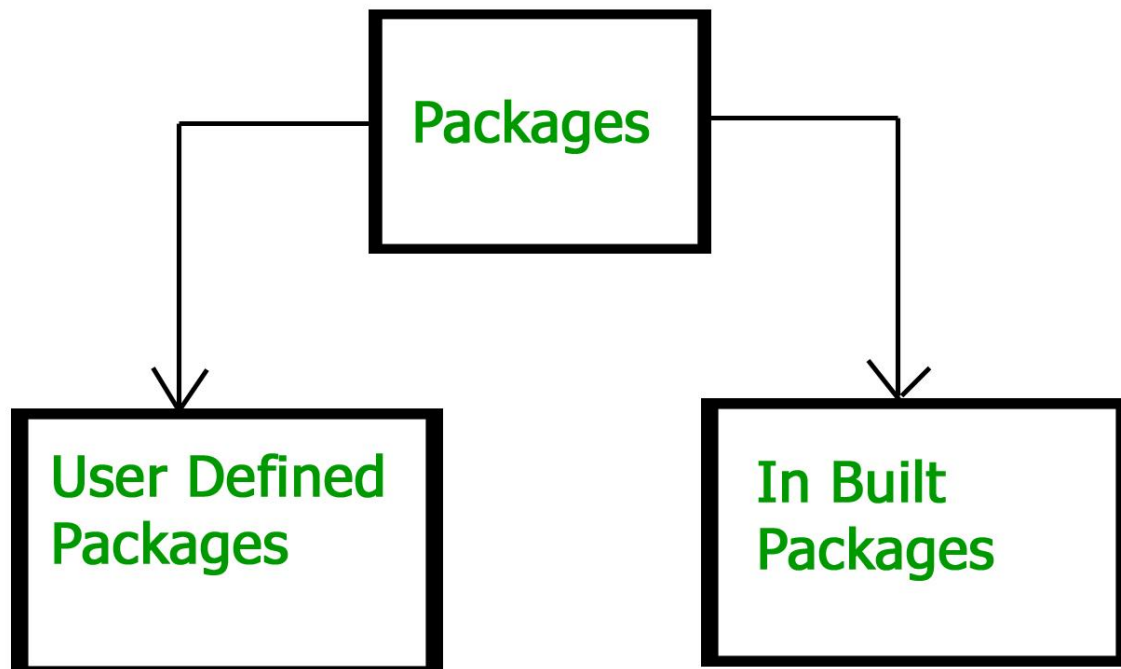
```

4.9 PACKAGES

Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces.

- Preventing naming conflicts. For example there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee
- Making searching/locating and usage of classes, interfaces, enumerations and annotations easier
- Providing controlled access: protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only.
- Packages can be considered as data encapsulation (or data-hiding).

Types of packages:



1. Built-in Packages

These packages consist of a large number of classes which are a part of Java API. Some of the commonly used built-in packages are:

- 1 **java.lang**
- 2 **java.io**
- 3 **java.util**
- 4 **java.applet**
- 5 **java.awt**
- 6 **java.net**

2.User-defined packages

These are the packages that are defined by the user. First we create a directory **myPackage** (name should be same as the name of the package). Then create the **MyClass** inside the directory with the first statement being the **package names**.

package access modifiers

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

EXAMPLE:

```
package mypackage;

public class packageDemo
{
    public void show()
    {
        System.out.println("this is package");
    }
}

class B extends packageDemo
{
    public static void main(String arg[])
    {
        B pd=new B();
        pd.show();
    }
}
```